Older GPU / DirectX Troubleshooting

What to do when Unity throws GPU fence or shader errors on older laptops

VIDEO GAME CREATION

When You'll Hit This

Some student and instructor laptops - especially older business/office laptops with modest integrated or low-end discrete GPUs - only properly support DirectX 10, with partial (incomplete) DirectX 11 support. Unity 6.x defaults to newer rendering paths that assume more complete DirectX 11/12 support than this hardware actually provides. The result is a machine that installs Unity fine, opens projects fine, and even runs the Hello World exercise successfully - but throws GPU-related warnings and errors in the Console along the way.

This was first documented while testing the Unity install on an older laptop with an NVIDIA NVS 3100M GPU (a low-end mobile workstation chip). The same symptoms are likely on any similarly dated hardware - Intel integrated graphics from before ~2012, or comparable low-end NVIDIA/AMD mobile chips of that era.

How to Recognize It

Two errors in the Unity Console are the telltale signs:

- `GpuFence::Create(): Failed to create ID3D11Fence, error 0x80070057`
- `Shader 'Hidden/Universal Render Pipeline/Edge Adaptive Spatial Upsampling' is not supported (in 'Blit FSR Upscaling'). PostProcessing render passes will not execute.`

If you see either of these - especially alongside a game that otherwise seems to run (the Hello World "Debug.Log" output still appears in Console) - this is very likely the cause, not a mistake in the exercise steps.

Fix 1: Adjust Settings in Unity (Stay on the Current Version)

This keeps you on the installed Unity version and works around the incompatible rendering features without downgrading anything.

Disable post-processing on the default render pipeline

1. Edit > Project Settings > Quality > Render Pipeline Asset.

2. Open the URP asset assigned there (its default name is something like "URP-...-Renderer" or the project's default Universal Render Pipeline / Renderer 2D Data asset).
3. In the Inspector for that Renderer 2D Data asset, find Post Processing and disable it.

Reorder the Graphics API so DirectX 11 (or OpenGL) is used instead of DirectX 12

4. Edit > Project Settings > Player > Other Settings > Graphics APIs for Windows.
5. Remove DirectX 12 support from the list entirely.
6. Move OpenGLCore (Deprecated) above DirectX 11 in the list, so it's tried first - or simply leave DirectX 11 as the top entry if OpenGL isn't offered as an option.

Restart the Unity Editor after changing the Graphics API list - Unity will prompt you to do this automatically.

Fix 2: Downgrade to Unity 2023 LTS

If Fix 1 doesn't fully resolve the errors, or if a student wants to do any 3D work at all, downgrading the project to Unity 2023.2 LTS (using DirectX 11) tends to be noticeably more stable on older hardware than Unity 6.x, and avoids the GPU fence error entirely.

When to Downgrade vs. Stay on Unity 6.x

Situation	Recommendation
Focused on 2D work (sprites, tilemaps, 2D physics)	Stay on Unity 6.x with OpenGL (Fix 1) - current setup works fine.
Want to try any 3D, even basic 3D	Downgrade to Unity 2023.2 LTS + DirectX 11.
Current setup runs without crashes	No need to change anything.
Want better stability generally on this hardware	Downgrade to Unity 2023.2 LTS + DirectX 11.

What Older Hardware Like This Can Still Do (3D)

On a low-end GPU like the NVS 3100M, running Unity 2023.2 LTS with DirectX 11, students can still learn real 3D fundamentals:

- Low-poly 3D modeling (simple shapes, basic meshes)
- Basic lighting (baked lightmaps, simple point/directional lights)
- Standard shaders (Shader Model 3.0-4.0, no compute shaders)
- Simple animations (rigged characters, basic mechanics)
- Basic physics (colliders, rigidbodies)
- URP rendering, without advanced post-processing or volumetrics

What's Off the Table (Hardware-Limited)

- HDRP (requires Shader Model 5.0+ and compute shaders)
- Real-time global illumination
- Advanced GPU-based particle systems
- High-poly models with complex (PBR-heavy) shaders

- VR/AR development

Is Hardware Like This Actually Usable for Teaching 3D?

Yes. Many successful indie games started on far weaker hardware than this. The formula that works within these limits:

- Low-poly, stylized art rather than photorealistic art
- Baked lighting rather than real-time lighting
- Efficient, simple shaders rather than PBR-heavy ones
- Small scenes rather than open-world scale

Unity 2023.2 LTS with DirectX 11 on hardware like the NVS 3100M handles all of the above for learning purposes. Students can complete tutorials, build small 3D prototypes, and learn the core concepts without hitting a hard wall.

Bottom Line

For 2D learning: stay on Unity 6.x + OpenGL (Fix 1) - the current setup already works.
For 3D learning: downgrade to Unity 2023.2 LTS + DirectX 11 (Fix 2) - better compatibility, no GPU fence errors.
Either way: older hardware will limit what a student can ship, but it will not block them from learning the fundamentals.

Hardware this old is "potato" by modern standards, but it is still fully capable of teaching game development basics. The real limit is what a project can ship, not what a student can learn.